LAB: USART - Bluetooth

Date: 2026-09-26

Author/Partner: Choi Jung Hyun

Github: repository link

Demo Video: Youtube link

PDF version:

Introduction

In this lab, we will learn how to configure and use 'USART(Universal synchronous asynchronous receiver transmitter)' of MCU. Then, we will learn how to communicate between your PC and MCU and MCU to another MCU with wired serial communication.

- **Mission 1**: Control LED(LD2) of other MCU using wired serial communication
- **Mission 2**: Control a stepper motor with wireless Bluetooth communication

You must submit

- LAB Report (*.md & *.pdf)
- Zip source files(main*.c, ecRCC.h, ecGPIO.h, ecSysTick.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - DC motor, DC motor driver(L9110s),
    - Bluetooth Module(HC-06),

Software

- Keil uVision, CMSIS, EC_HAL library

Tutorial: USART with TeraTerm

Learn how to use USART library and the serial monitor TeraTerm

{% content-ref url="/pages/gMZ3F9dPug4zL3Uy10CQ" %} Tutorial: USART with TeraTerm {% endcontent-ref %}

Problem 0: USART library

Download the following files:

- ecUART2_student.c
- ecUART2_student.h

Then, change their names as

- ecUART2.c
- ecUART2.h

> Header files must locate in `EC\include\`. Fill-in the comment box.
>
> Update ecSTM32F4v2.h

Exercise

1. Fill in the blanks in the library files.
2. Then, run the test code in different baud rate to validate your code.

   * Tera-Term Terminal: (1) Baud 9600, (2) Baud 57600
3. You may do the exercise after you complete this Lab

ecUSART2.h

{% code expandable="true" %}

```
#define BAUD_9600   9600
#define BAUD_38400  38400
#define BAUD_57600  57600
#define BAUD_115200 115200

/*================ EXERCISE: Fill In the Blanks ================*/
/*================ General Configuration ================*/
// General configuration
void USART_init(USART_TypeDef* USARTx, PinName_t pin_GPIO_TX, PinName_t
pin_GPIO_RX, uint32_t baud);    // copy of USART_setting
void USART_baud(USART_TypeDef* USARTx, uint32_t baud);
void USART_pins(USART_TypeDef* USARTx, PinName_t pin_GPIO_TX, PinName_t
pin_GPIO_RX);
void USART1_baud(uint32_t baud);
void USART2_baud(uint32_t baud);

// Example:
//  For BT serial : specific RX/TX pins
//  USART_init(USART1, PA_9,PA_10, BAUD_9600);  // PA9 - RXD , PA10 - TXD


/*================ PROVIDED  DON'T MODIFY HERE ================*/
/*================ Configuration for each USART ================*/
// Configuration UART 1, 2 using default settings
void USART1_init(void);
```

```
void USART2_init(void);

// USART write & read
void USART1_write(uint8_t* buffer, uint32_t nBytes);
void USART2_write(uint8_t* buffer, uint32_t nBytes);
uint8_t USART1_read(void);
uint8_t USART2_read(void);

// RX Inturrupt Flag USART1,2
uint32_t is_USART1_RXNE(void);
uint32_t is_USART2_RXNE(void);

/*================ General Configuration ================*/
void USART_clock_enable(USART_TypeDef* USARTx);
void USART_write(USART_TypeDef* USARTx, uint8_t* buffer, uint32_t nBytes);
uint8_t USART_read(USART_TypeDef * USARTx);
uint32_t is_USART_RXNE(USART_TypeDef * USARTx);
void USART_delay(uint32_t us);
```

{% endcode %}

TU_USART_testrun.c

```
#include <stdio.h>
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecUART2.h"


uint8_t PC_string[]="MCU Initialized \r\n";

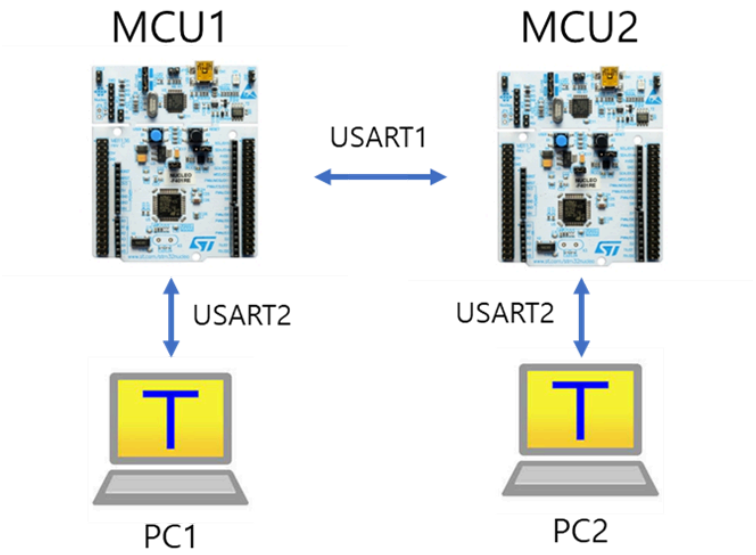
// Initialiization
void setup(void)
{
    // MCU_init();
    RCC_PLL_init();
    SysTick_init();
    // USART2: PA_2=TX, PA_3=RX, 57600 baud
    USART_init(USART2, PA_2, PA_3, BAUD_57600);
}


int main(void) {
    // Initialiization ---------------------------------------------------------
    setup();
  USART2_write(PC_string, sizeof(PC_string));

  uint32_t lineNumber = 0;
  // Inifinite Loop ----------------------------------------------------------
    while (1){
        printf("Line %lu >>\r\n", lineNumber);
```

```
        delay_ms(2000);
        lineNumber++;
    }
}
```

Problem 1: Communicate MCU1-MCU2 using RS-232



Procedure

1. Create a new project under the directory `EC\LAB\`

- The environment name is "**LAB_USART_LED".**
- Create a new source files named as "**LAB_USART_LED.c"**

> You MUST write your name on the source file inside the comment section.

2. Make sure you updated the library in `\EC\include\`

- **ecUART2.h, ecUART2.c**
- Also, update **ecSTM32F4v2.h**

3. Connect each MCUs to each PC with **USART 2** via USB cable (ST-Link)

- MCU1 to PC1
- MCU2 to PC2

4. Connect MCU1 to MCU2 with **USART 1**

- Connect RX/TX pins externally with jumper wires as
  - MCU1_TX to MCU2_RXD
  - MCU1_RX to MCU2_TX

5. Connect **GND pins** between MCU1-MCU2

{% hint style="info" %} MUST connect the same GND pin between MCU1 and MCU2 {% endhint %}

6. Send a message from PC_1 by typing keys on Teraterm.

> You have to press "Enter" to end the message.

7. The received message by MCU_2 should be displayed on PC_2.
8. Turn other MCU's LED(LD2) ON/OFF by :

- Press key "**L**" to Turn OFF LED
- Press key "**H**" to Turn ON LED

Configuration

Function	**Port - Pin**	**Configuration**
System Clock		PLL 84MHz
USART2 : USB cable (ST-Link)		No Parity, 8-bit Data, 1-bit Stop bit, 9600 baud-rate
USART1 : MCU1, MCU2	TXD: PA9 RXD: PA10	No Parity, 8-bit Data, 1-bit Stop bit, 9600 baud-rate
Digital Out: LD2	PA5	

Example Code

Example codes for communicating between two MCUs

Example 1 : RX/TX **one character** at a time

- Receive (Interrupt), Send (Polling)
- USART2:
  - Sending a character from PC_keyboard to MCU
  - Sending a character from MCU to PC serial monitor (**Echo keyboard type**)
- USART1:
  - Send and Receive a character from other MCU
- To Display the received message on PC monitor
  - MCU_2:USART1 -> MCU_1:USART1 -> MCU_1:USART2-> Serial monitor_1

Example 2 : RX/TX a string

```
#include "ecSTM32F4v2.h"// this should include #include "ecUART2.h"
//#include "ecUART2.h"
```

```
static volatile uint8_t PC_Data = 0;
static volatile uint8_t BT_Data = 0;
uint8_t PC_string[]="Loop:\r\n";

void setup(void){
    RCC_PLL_init();
    SysTick_init();

    // USART2: USB serial init
    USART2_init();  // default 9600
    //USART2_baud(BAUD_38400);

    // USART1: MCU2 serial init
    USART1_init(); // default 9600
    //USART1_baud(BAUD_38400);
}

int main(void){
    setup();
    printf("MCU Initialized\r\n");
    while(1){
        // USART Receive: Use Interrupt only
        // USART Transmit: Polling
        //USART2_write(PC_string, 7);
        delay_ms(2000);
    }
}

void USART2_IRQHandler(){       // USART2 RX Interrupt
    if(is_USART2_RXNE()){
        PC_Data = USART2_read();        // RX from UART2 (PC)
        USART2_write(&PC_Data,1);       // TX to USART2  (PC)    Echo of keyboard
typing
        USART1_write(&PC_Data,1);     // TX to Other MCU
    }
}

void USART1_IRQHandler(){           // USART2 RX Interrupt
    if(is_USART1_RXNE()){
        BT_Data = USART1_read();                // RX from other MCU UART1
        printf("RX: %c \r\n",BT_Data);  // TX to USART2(PC)
    }
}
```

```
#include "ecSTM32F4v2.h"  // this should include #include "ecUART2.h"

#define MAX_BUF 10
#define END_CHAR 13

static volatile uint8_t buffer[MAX_BUF]={0, };
static volatile uint8_t PC_string[MAX_BUF]={0, };
```

```c
static volatile uint8_t PC_data = 0;

static volatile int idx = 0;
static volatile int bReceive =0;

void setup(void){
    RCC_PLL_init();
    SysTick_init();

    // USART2: USB serial init
    USART2_init();
    //USART2_baud(BAUD_38400);
    // USART1: MCU2 serial init
    USART1_init();
    //USART1_baud(BAUD_38400);
}

int main(void){
    setup();
    printf("MCU Initialized\r\n");

    while(1){
        if (bReceive == 1){
            printf("PC_string: %s\r\n", PC_string);
            bReceive = 0;
        }
    }
}

void USART2_IRQHandler(){       // USART2 RX Interrupt
    if(is_USART2_RXNE()){
        PC_data = USART2_read();     // RX from UART2 (PC)
        USART2_write(&PC_data,1);    // TX to USART2 (PC) Echo of keyboard typing

        // Creates a String from received serial character
        if(PC_data != END_CHAR && (idx < MAX_BUF)){
            buffer[idx] = PC_data;
            idx++;
        }
        else if (PC_data== END_CHAR) {
            bReceive = 1;
            // reset PC_string
            memset(PC_string, 0, sizeof(char) * MAX_BUF);
            // copy to PC_string
            memcpy(PC_string, buffer, sizeof(char) * idx);
            // reset buffer
            memset(buffer, 0, sizeof(char) * MAX_BUF);
            idx = 0;
            USART1_write(PC_string,sizeof(PC_string)); // TX to other MCU
        }
        else{       // if(idx >= MAX_BUF)
            idx = 0;
            // reset PC_string
            memset(PC_string, 0, sizeof(char) * MAX_BUF);
```

```
            // reset buffer
            memset(buffer, 0, sizeof(char) * MAX_BUF);
            printf("ERROR : Too long string\r\n");
        }
    }
}

// Added: receive a string from the other MCU through PA9/PA10.
void USART1_IRQHandler(){
    if(is_USART1_RXNE()){
        PC_data = USART1_read();
        USART2_write(&PC_data,1); // Echo received character to local PC

        if(PC_data != END_CHAR && (idx < MAX_BUF)){
            buffer[idx] = PC_data;
            idx++;
        }
        else if (PC_data == END_CHAR){
            bReceive = 1;
            memset(PC_string, 0, sizeof(char) * MAX_BUF);
            memcpy(PC_string, buffer, sizeof(char) * idx);
            memset(buffer, 0, sizeof(char) * MAX_BUF);
            idx = 0;
        }
        else{
            idx = 0;
            memset(PC_string, 0, sizeof(char) * MAX_BUF);
            memset(buffer, 0, sizeof(char) * MAX_BUF);
            printf("ERROR : Too long string\r\n");
        }
    }
}
```

Code

Your code goes here

Explain your source code with necessary comments.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Choi JungHyun
Created          : 10-08-2026
Modified         : 10-19-2026
Language/ver     : C in VS Code

Description      : Bidirectional USART2 (PC) & USART1 (Inter-MCU) LED Control
/----------------------------------------------------------------*/

#include "ecSTM32F4v2.h" // Includes ecUART2.h, ecGPIO2.h, ecRCC2.h
```

```
#define LD2_PIN PA_5

static volatile uint8_t PC_Data = 0;
static volatile uint8_t BT_Data = 0;
uint8_t PC_string[] = "Loop:\r\n";

void setup(void){
    RCC_PLL_init();
    SysTick_init();

    // Configure LD2 Pin (PA_5) as Output
    GPIO_init(LD2_PIN, OUTPUT);
    GPIO_otype(LD2_PIN, EC_PUSH_PULL);
    GPIO_pupd(LD2_PIN, EC_NONE);
    GPIO_ospeed(LD2_PIN, EC_HIGH_SPEED);
    GPIO_write(LD2_PIN, LOW); // Start with LD2 OFF

    // USART2: USB serial init
    USART2_init(); // default 9600
    //USART2_baud(BAUD_38400);

    // USART1: MCU2 serial init
    USART1_init(); // default 9600
    //USART1_baud(BAUD_38400);
}

int main(void){
    setup();
    printf("MCU Initialized\r\n");
    while(1){
        // USART Receive: Use Interrupt only
        // USART Transmit: Polling
        //USART2_write(PC_string, 7);
        delay_ms(2000);
    }
}

/*-----------------------------------------------------------------\
        INTERRUPT SERVICE ROUTINES (ISRs)
/-----------------------------------------------------------------*/

// USART2 RX Interrupt: Input from PC terminal
void USART2_IRQHandler(void){
    if(is_USART2_RXNE()){
        PC_Data = USART2_read();        // RX from UART2 (PC)
        USART2_write(&PC_Data, 1);      // TX to USART2 (PC) - Echo of keyboard
typing
        USART1_write(&PC_Data, 1);      // TX to Other MCU over USART1
    }
}

// USART1 RX Interrupt: Input from Other MCU
void USART1_IRQHandler(void){
```

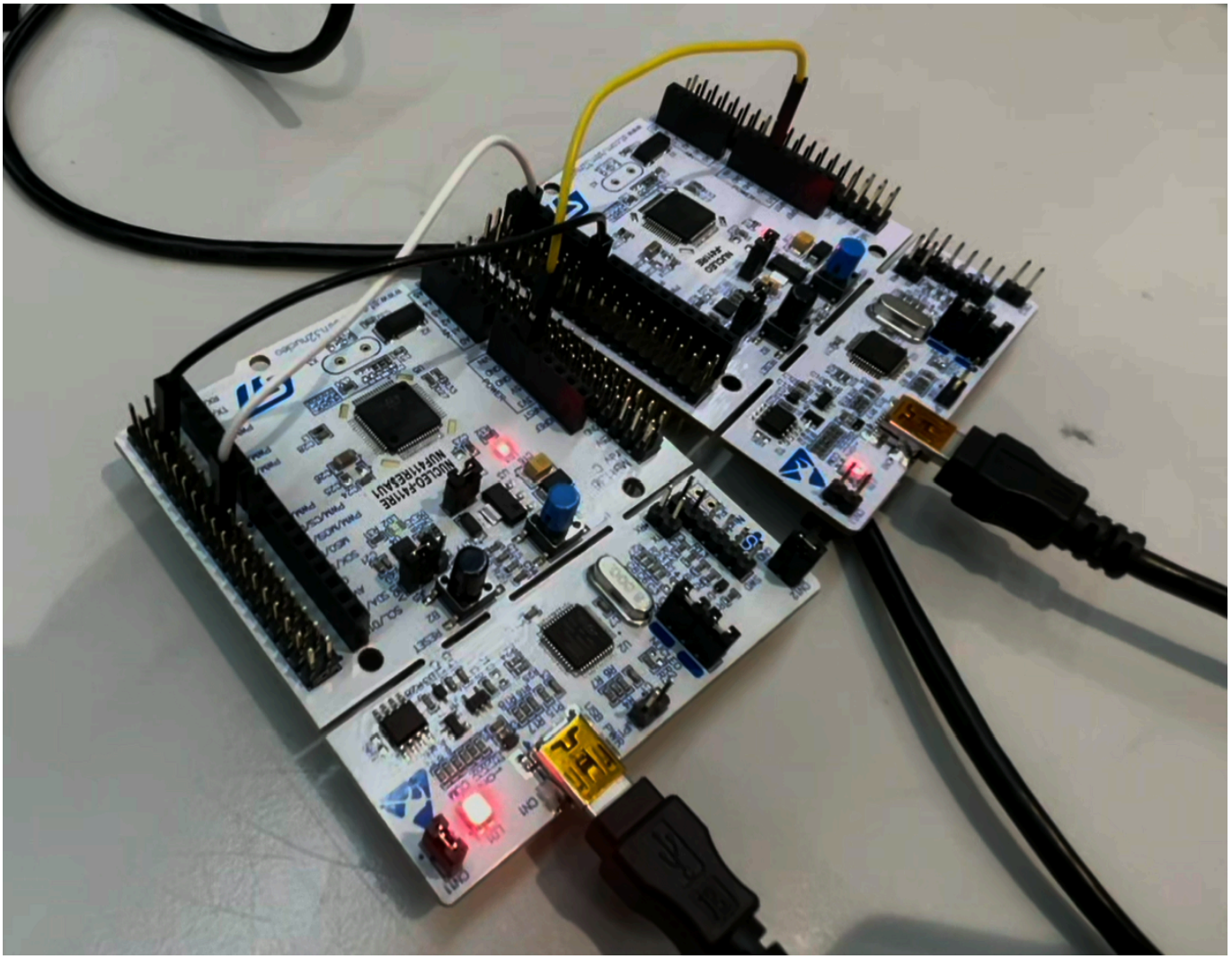
```
        if(is_USART1_RXNE()){
            BT_Data = USART1_read();            // RX from other MCU UART1
            printf("RX: %c \r\n", BT_Data);    // TX to USART2 (PC)

            // Control LD2 on this MCU based on received command from other MCU
            if (BT_Data == 'H' || BT_Data == 'h') {
                GPIO_write(LD2_PIN, HIGH);      // Turn ON LD2
                printf("LD2: ON\r\n");
            }
            else if (BT_Data == 'L' || BT_Data == 'l') {
                GPIO_write(LD2_PIN, LOW);       // Turn OFF LD2
                printf("LD2: OFF\r\n");
            }
        }
    }
```

Result

Experiment images and results

> Show experiment images /results



Add demonstation video link

Discussion

Analyze the result and explain any other necessary discussion.

Problem 2: Control Stepper Motor via Bluetooth



Tutorial: Bluetooth

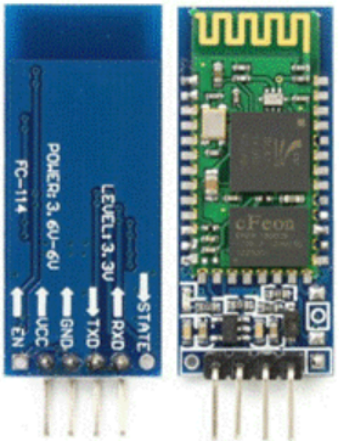
Follow the tutorial for Bluetooth setting.

{% content-ref url="/pages/0hcwqpOQ2L0RzdlCgqES" %} Tutorial: Bluetooth {% endcontent-ref %}

Search for the bluetooth module specification sheet (HC-06) and study the pin configurations.

- The default PIN number is 1234.

Example of connecting to USART1



Bluetooth Module (HC-06)	STM32F411RE
RxD	PA_9(UART1_TX)
TxD	PA_10(UART1_RX)
GND	GND
VCC	5V

Procedure

1. Create a new project

- The environment name is "**LAB_USART_Bluetooth".**
- Create a new source files named as "**LAB_USART_Bluetooth.c"**

> You MUST write your name on the source file inside the comment section.

2. Make sure your updated library in `include\` to your project.

- **ecUART2.h, ecUART2.c**
- Update **ecSTM32F4v2.h**

3. Connect the MCU-BT module. Use USART 1

- Connect RX/TX pins as
  - **MCU_TX to BT_RX**
  - **MCU_ RX to BT_TX**

4. Connect the BT module (HC-06) to the PC

5. Check the Bluetooth connection by turning MCU's LED(LD2) ON/OFF by sending text of "**L**" or "**H**" from the PC (serial terminal)
6. Connect a stepper motor to MCU (**see Lab: Stepper**)
7. Control Step motor by the Keyboard inputs

- Rotate the stepper motor 2 revolutions with an appropriate speed
- Press 'J' : To run CW direction
- Press 'K' : To run CCW direction
- Press 'P' : Emergency STOP

You may use other the key inputs as your preference.

Configuration

Function	Port - Pin	Configuration
System Clock		PLL 84MHz

Function	Port - Pin	Configuration
USART1 : MCU -Bluetooth	TXD: PA9 RXD: PA10	No Parity, 8-bit Data, 1-bit Stop bit, 9600 baud-rate
Digital Out: LD2	PA5	
Motor Pin A, B, A', B'	PB10, PB4, PB5, PB3	DOUT

Code

Your code goes here

Explain your source code with necessary comments.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Choi JungHyun
Created          : 10-08-2026
Modified         : 10-26-2026
Language/ver     : C in VS Code

Description      : Problem 2 - bt chip stepper control(j-clockwise, k-ccw, p -
emergency stop) - problem2
/----------------------------------------------------------------*/

#include "ecSTM32F4v2.h"
#include "ecStepper.h"

#define LD2_PIN PA_5

// pins connected to the stepper
PinName_t pinStepper[4] = {PB_10, PB_4, PB_5, PB_3};

static volatile uint8_t BT_Data = 0;
static volatile uint8_t cmd_flag = 0; // 0: stop, 1: CW, 2: CCW

void setup(void){
    RCC_PLL_init();            // 84 MHz
    SysTick_init();            // delay timer

    // Onboard LED conditions
    GPIO_init(LD2_PIN, OUTPUT);
    GPIO_otype(LD2_PIN, EC_PUSH_PULL);
    GPIO_pupd(LD2_PIN, EC_NONE);
    GPIO_ospeed(LD2_PIN, EC_HIGH_SPEED);
    GPIO_write(LD2_PIN, LOW);
```

```
    // stepper Motor step mode and speed
    stepper_init(HALF, pinStepper);
    stepper_speed(15);

    // USART2(PC USB) & USART1(HC-06)
    USART2_init();
    USART_init(USART1, PA_9, PA_10, BAUD_9600); // baud rate, tx rx pins
}

int main(void) {
    setup();
    printf("MCU Bluetooth Stepper Control Ready\r\n");

    while (1) {
        // run CW direction 2 Full revolutions
        if (cmd_flag == 1) {
            cmd_flag = 0;    // Clear flag
            printf("Running CW: 2 Revolutions...\r\n");

            // 8192 steps = 2 full rotations in half step mode
            stepper_step(8192, DIR_CW);

            // in case p was pressed restart motor from that position
            stepper_resume();
            printf("CW Motion Completed/Finished.\r\n");
        }

        // CCW Direction 2 full revolutions
        else if (cmd_flag == 2) {
            cmd_flag = 0;    // clear flag
            printf("Running CCW: 2 Revolutions...\r\n");

            //
            stepper_step(8192, DIR_CCW);

            // Restart motor at stopped position
            stepper_resume();
            printf("CCW Motion Completed/Finished.\r\n");
        }
    }
}


void USART1_IRQHandler(void){
    if (is_USART1_RXNE()) {
        BT_Data = USART1_read();     // read character over bluetooth
        USART1_write(&BT_Data, 1);   // send back to bluetooth Tera

        // led2 toggle feature
        if (BT_Data == 'H' || BT_Data == 'h') {
            GPIO_write(LD2_PIN, HIGH);
        }
        else if (BT_Data == 'L' || BT_Data == 'l') {
```

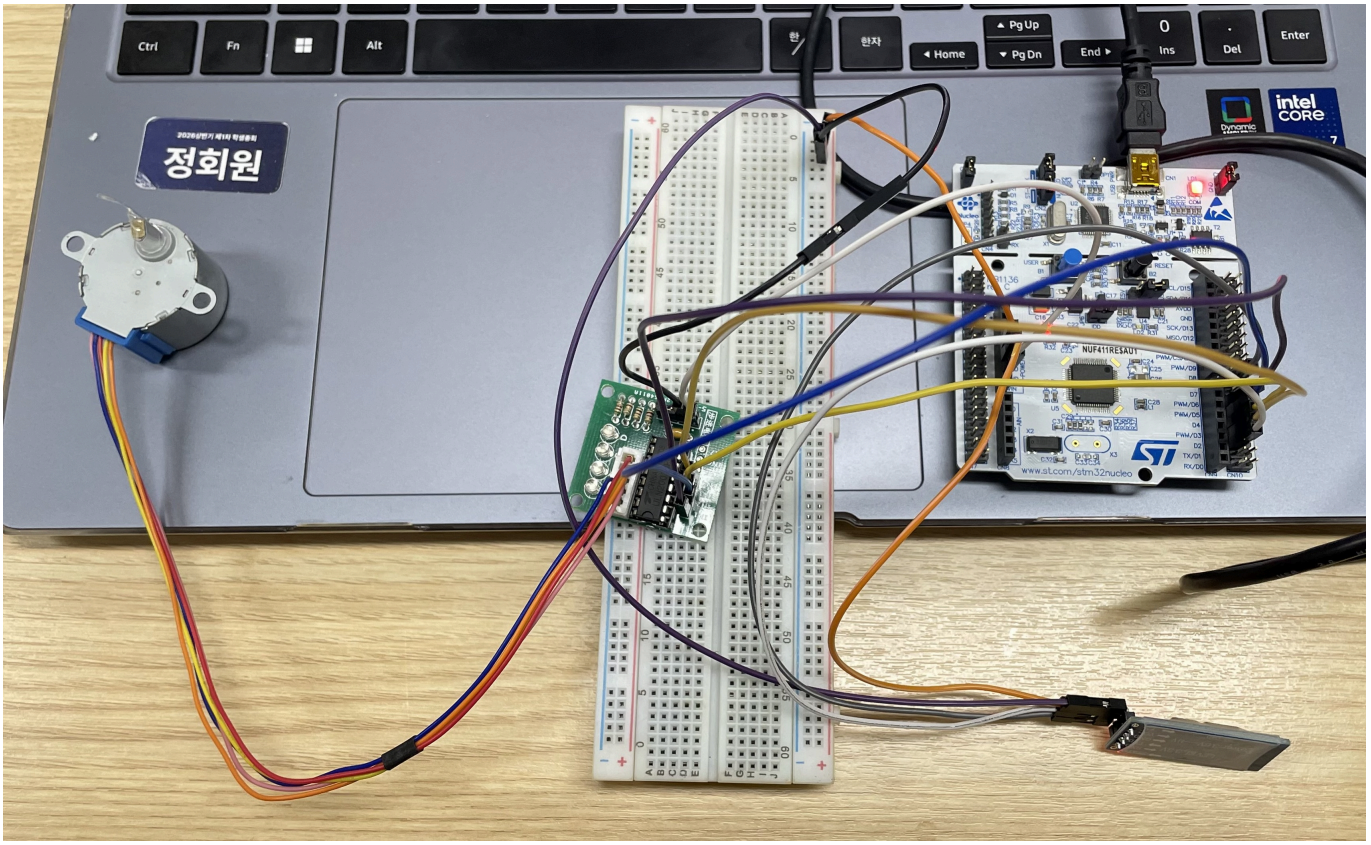
```
            GPIO_write(LD2_PIN, LOW);
        }

        // stepper control commands
        else if (BT_Data == 'J' || BT_Data == 'j') {
            stepper_resume(); // motor run cw
            cmd_flag = 1;
        }
        else if (BT_Data == 'K' || BT_Data == 'k') {
            stepper_resume();
            cmd_flag = 2;     // motor run ccw
        }
        else if (BT_Data == 'P' || BT_Data == 'p') {
            cmd_flag = 0;               // Clear motion commands
            stepper_emergency_stop();   // emergency stop in ISR
            printf("\r\n[BT] EMERGENCY STOP ACTIVATED!\r\n");
        }
    }
}
```

Result

Experiment images and results

> Show experiment images /results



demonstation video link

Discussion

Analyze the result and explain any other necessary discussion.

In serial communication, characters (k,l,j,p,h) are processed inside non blocking interrupt(USART1_IRQHandler), while the main code handles the motor drive routines. At 9600baud in half-step mode, the stepper motor requires 8192 steps for 2 full revolutions. coding stepper_emergency_stop() directly inside the ISR instantly cuts power to the motor, providing emergency stopping mid-rotation without getting blocked by delay loops.

Reference

Complete list of all references used (github, blog, paper, etc)

```
```

Troubleshooting

Does not show a float number in printf()

In `platformio.ini`, modify

```
build_flags = -Wl,-u,_printf_float -Wl,-u,_scanf_float -std=c11 -O3 -g3
```

```
# NOTE: the newlib-nano symbols are _printf_float / _scanf_float. Without them
# --specs=nano.specs links a printf that silently prints nothing for %f.
```